

# Explanation - Vedant Bhatnagar

Reliable Data Transfer over UDP (CS-30003 CA1, P3)

## Member: Vedant Bhatnagar (24155892) | Part: Selective Repeat, adaptive RTO (Jacobson/Karels), Karn's algorithm

Your files: vedant\_bhatnagar/selective\_repeat.py, vedant\_bhatnagar/rto.py

### *The 1-minute story*

- **Selective Repeat (SR)** is the smartest of our three protocols. Like Go-Back-N it has a window, but **only the lost packet is resent**, never the good ones.
- How? (1) Every packet gets its **own ACK**. (2) Every packet has its **own timer**. (3) The receiver has a **buffer**: out-of-order packets are kept and delivered once the gap is filled.
- **RTO** (Retransmission TimeOut) = how long to wait for an ACK. A fixed number is bad: too short = needless resends, too long = slow recovery. So we **measure the round-trip time (RTT)** and adapt.
- **Jacobson/Karels**: keep a smoothed average RTT (srtt) and a smoothed wobble (rttvar).  $RTO = srtt + 4 \times rttvar$ . This is what real TCP does.
- **Karn's algorithm**: (a) never measure RTT from a packet that was resent, because you can't tell whether the ACK answers the first or the second copy; (b) after a timeout, double the RTO (exponential backoff) so we don't flood a struggling network.

### *Example: GBN vs SR with one loss*

Window 4, packets 0-3 sent, packet 1 lost.

**GBN**: receiver discards 2 and 3; timer fires; sender resends 1, 2 AND 3 (3 packets).

**SR**: receiver ACKs 0, 2 and 3 and keeps 2, 3 in its buffer; only packet 1's timer fires; sender resends ONLY 1 (1 packet); receiver then delivers 1, 2, 3 in order.

### *The RTO formulas*

First sample:  $srtt = sample$ ,  $rttvar = sample / 2$

Later samples:  $rttvar = 0.75 \times rttvar + 0.25 \times |srtt - sample|$  ;  $srtt = 0.875 \times srtt + 0.125 \times sample$  ;  $rto = srtt + 4 \times rttvar$

(0.125 = 1/8 is ALPHA, 0.25 = 1/4 is BETA). Then RTO is clamped between 50 ms and 2 s.

### *Note about the experiments (mention if asked)*

The proposal says 'RTO tuning is held constant as a control'. So in the experiments we switch adaptive RTO OFF (adaptive=False) and all three protocols use the same fixed timeout. Then any difference in goodput comes from the protocol design (selective retransmission), not from a smarter timer. In the normal command-line run adaptive RTO is ON.

### *Variables to remember*

- **acked[i]** = has packet i been acknowledged? **sent\_at[i]** = when packet i was last sent (its own timer). **resent[i]** = was it ever retransmitted (for Karn).
- **base** = oldest un-acked packet, **next\_seq** = next new packet. **rcv\_base** (receiver) = lowest packet not yet delivered. **buffer** = dictionary of out-of-order packets.

## File: vedant\_bhatnagar/rto.py

| Code                                                                                                          | What it means (plain English)                                                                                                   |
|---------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| <code>class RTOEstimator:</code>                                                                              | A small class that remembers the RTO state and updates it.                                                                      |
| <code>    ALPHA = 0.125 # weight of the new sample for srtt (1/8)</code>                                      | Weight for new samples in the average (1/8): the average changes slowly.                                                        |
| <code>    BETA = 0.25 # weight of the new sample for rttvar (1/4)</code>                                      | Weight for new samples in the variation estimate (1/4).                                                                         |
| <code>    MIN_RTO = 0.05 # never go below 50 ms</code><br><code>    MAX_RTO = 2.0 # never go above 2 s</code> | Lower and upper limits (50 ms and 2 s) so the timer never becomes silly.                                                        |
| <code>    def __init__(self, initial_rto, adaptive=True):</code>                                              | Constructor. <code>initial_rto</code> = starting timeout; <code>adaptive</code> = should we learn?                              |
| <code>        self.value = initial_rto # the current RTO in seconds</code>                                    | The current RTO value that the sender uses.                                                                                     |
| <code>        self.adaptive = adaptive # if False, RTO stays fixed</code>                                     | If False, the RTO never changes (used in the fair-comparison experiments).                                                      |
| <code>        self.srtt = None</code><br><code>        self.rttvar = None</code>                              | Smoothed RTT and variation. None = we haven't measured anything yet.                                                            |
| <code>    def update(self, sample):</code>                                                                    | Called each time we get a clean RTT sample.                                                                                     |
| <code>        if not self.adaptive:</code><br><code>            return</code>                                 | Fixed-RTO mode: do nothing.                                                                                                     |
| <code>        if self.srtt is None: # very first sample</code>                                                | Is this the very first measurement?                                                                                             |
| <code>            self.srtt = sample</code><br><code>            self.rttvar = sample / 2</code>              | First time: start the average at the sample and the variation at half of it (rule from RFC 6298).                               |
| <code>        else:</code>                                                                                    | For all later samples:                                                                                                          |
| <code>            self.rttvar = (1 - self.BETA) * self.rttvar + self.BETA * abs(self.srtt - sample)</code>    | Update the wobble: 75% old value + 25% of how far the new sample is from the average. (This must be done BEFORE updating srtt.) |
| <code>            self.srtt = (1 - self.ALPHA) * self.srtt + self.ALPHA * sample</code>                       | Update the average: 87.5% old + 12.5% new sample. A weighted moving average.                                                    |
| <code>            rto = self.srtt + 4 * self.rttvar</code>                                                    | Jacobson/Karels formula: average plus four times the wobble. The 4x gives a safety margin so we rarely time out early.          |
| <code>            self.value = min(max(rto, self.MIN_RTO), self.MAX_RTO)</code>                               | Clamp to [50 ms, 2 s]: <code>max()</code> enforces the floor, <code>min()</code> enforces the ceiling.                          |
| <code>    def backoff(self):</code>                                                                           | Called when a timeout happens.                                                                                                  |
| <code>        if not self.adaptive:</code><br><code>            return</code>                                 | Fixed mode: do nothing.                                                                                                         |
| <code>        self.value = min(self.value * 2, self.MAX_RTO)</code>                                           | KARN rule (b): double the RTO (but never above 2 s). The next clean sample will bring it back to a normal value.                |

## File: vedant\_bhatnagar/selective\_repeat.py

| Code                                                                                                       | What it means (plain English)                                                                                                      |
|------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <code>import time</code>                                                                                   | For timers.                                                                                                                        |
| <code>from packet import make_packet, receive_packet, DATA, ACK, FIN, GIVE_UP_SECONDS</code>               | Reuse Arijeet's packet tools.                                                                                                      |
| <code>from rto import RTOEstimator</code>                                                                  | Reuse my RTO class.                                                                                                                |
| <code>def send(chan, dest, chunks, window, timeout, adaptive=True):</code>                                 | SR sender. timeout is the starting RTO; adaptive says whether RTO learns.                                                          |
| <code>    items = [(DATA, c) for c in chunks] + [(FIN, b'')]</code>                                        | DATA packets plus final FIN.                                                                                                       |
| <code>    total = len(items)</code>                                                                        | How many packets in total.                                                                                                         |
| <code>    packets = [make_packet(t, seq, p) for seq, (t, p) in enumerate(items)]</code>                    | Pre-build all packets' bytes so resending is easy.                                                                                 |
| <code>    rto = RTOEstimator(timeout, adaptive) # adaptive timeout (Jacobson/Karels + Karn)</code>         | Create the RTO estimator.                                                                                                          |
| <code>    acked = [False] * total # has packet i been acknowledged?</code>                                 | A checklist: <code>acked[i]</code> is True once packet <code>i</code> is acknowledged. This is the 'selective' bookkeeping.        |
| <code>    sent_at = [0.0] * total # when packet i was last sent</code>                                     | For each packet, the time it was last sent. This gives every packet its own timer.                                                 |
| <code>    resent = [False] * total # was packet i ever retransmitted?</code>                               | Marks packets that were retransmitted (needed for Karn rule (a)).                                                                  |
| <code>    base = 0 # oldest unacknowledged packet<br/>    next_seq = 0 # next new packet to send</code>    | Window edges, as in GBN.                                                                                                           |
| <code>    packets_sent = 0<br/>    retransmissions = 0<br/>    last_progress = time.time()</code>          | Counters and the give-up safety timer.                                                                                             |
| <code>    while base &lt; total:</code>                                                                    | Until all packets are acknowledged.                                                                                                |
| <code>        while next_seq &lt; total and next_seq &lt; base + window:</code>                            | STEP 1, fill the window with new packets.                                                                                          |
| <code>            chan.sendto(packets[next_seq], dest)</code>                                              | Send packet <code>next_seq</code> .                                                                                                |
| <code>            sent_at[next_seq] = time.time()</code>                                                   | Start ITS OWN timer by remembering the send time.                                                                                  |
| <code>            packets_sent += 1<br/>            next_seq += 1</code>                                   | Count it and move the window's right edge.                                                                                         |
| <code>        earliest = min(sent_at[s] + rto.value for s in range(base, next_seq) if not acked[s])</code> | STEP 2, when will the first timer expire? For each unacked packet in the window, expiry time = sent time + RTO; take the smallest. |
| <code>        wait = max(0.001, earliest - time.time())</code>                                             | How long we can wait for an ACK before some timer fires (at least 1 ms so we never wait 'zero').                                   |
| <code>        reply = receive_packet(chan, wait)</code>                                                    | Wait for an ACK for that long.                                                                                                     |

| Code                                                                                                                          | What it means (plain English)                                                                                                                 |
|-------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <code>if reply is not None:</code>                                                                                            | An ACK (or some good packet) arrived.                                                                                                         |
| <code>reply_type, seq, _ = reply</code>                                                                                       | Unpack. In SR the number in an ACK is the sequence number of ONE packet (not cumulative).                                                     |
| <code>if reply_type == ACK and base &lt;= seq &lt; next_seq and not<br/>acked[seq]:</code>                                    | Accept only if: it is an ACK, the number is inside the window, and we haven't already recorded it (ignores duplicate ACKs).                   |
| <code>acked[seq] = True # mark just this packet as done</code>                                                                | Tick that packet off the checklist. Only THIS packet is confirmed.                                                                            |
| <code>last_progress = time.time()</code>                                                                                      | Progress was made.                                                                                                                            |
| <code>if not resent[seq]: # Karn (a): only clean samples<br/>rto.update(time.time() - sent_at[seq])</code>                    | KARN rule (a): only if the packet was never resent do we trust the sample. RTT = now - send time. Feed it to the estimator (Jacobson/Karels). |
| <code>while base &lt; total and acked[base]: # slide window over<br/>finished packets<br/>base += 1</code>                    | Slide the window forward over every packet at the left that is already acked (there could be several ticked in a row).                        |
| <code>now = time.time()<br/>timed_out = False</code>                                                                          | STEP 3: check the timers. Remember current time; timed_out = did any timer expire in this pass?                                               |
| <code>for seq in range(base, next_seq):</code>                                                                                | Look at every packet in the window.                                                                                                           |
| <code>if not acked[seq] and now - sent_at[seq] &gt;= rto.value:</code>                                                        | Not acknowledged AND its own timer has run out?                                                                                               |
| <code>chan.sendto(packets[seq], dest)</code>                                                                                  | Resend ONLY this packet (this is the 'selective' part).                                                                                       |
| <code>sent_at[seq] = now<br/>resent[seq] = True</code>                                                                        | Restart its timer and mark it as retransmitted (so Karn ignores its RTT).                                                                     |
| <code>packets_sent += 1<br/>retransmissions += 1<br/>timed_out = True</code>                                                  | Count it and remember that a timeout occurred.                                                                                                |
| <code>if timed_out:<br/>rto.backoff() # Karn (b): double the RTO</code>                                                       | KARN rule (b): after a timeout, double the RTO (once per pass, not once per packet).                                                          |
| <code>if now - last_progress &gt; GIVE_UP_SECONDS:<br/>raise TimeoutError("selective-repeat: receiver not responding")</code> | Safety net: no progress in 20 s -> error.                                                                                                     |
| <code>return {"packets_sent": packets_sent, "retransmissions":<br/>retransmissions,<br/>"final_rto": rto.value}</code>        | Return the counters plus the final RTO value (nice to show in the viva).                                                                      |
| <code>def receive(chan, dest, window, linger):</code>                                                                         | SR receiver. It needs `window` because it accepts packets within a window.                                                                    |
| <code>rcv_base = 0 # lowest sequence number not yet delivered</code>                                                          | Lowest packet number not yet delivered to the file.                                                                                           |
| <code>buffer = {} # out-of-order packets waiting: seq -&gt; (type, payload)</code>                                            | A dictionary: sequence number -> (type, payload) for out-of-order packets we are keeping.                                                     |
| <code>chunks = []<br/>finished = False</code>                                                                                 | In-order file pieces and the FIN flag.                                                                                                        |

| Code                                                                                                                                              | What it means (plain English)                                                                                  |
|---------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| <code>while True:</code>                                                                                                                          | Receive loop.                                                                                                  |
| <code>wait = linger if finished else GIVE_UP_SECONDS</code>                                                                                       | After FIN only linger briefly (to re-ACK a resent FIN).                                                        |
| <code>packet = receive_packet(chan, wait)</code>                                                                                                  | Get the next good packet.                                                                                      |
| <code>if packet is None:<br/>    if finished:<br/>        return chunks<br/>    raise TimeoutError("selective-repeat: sender went silent")</code> | Silence after FIN = done. Silence before FIN = error.                                                          |
| <code>packet_type, seq, payload = packet</code>                                                                                                   | Unpack.                                                                                                        |
| <code>if packet_type == ACK:<br/>    continue</code>                                                                                              | Ignore ACKs.                                                                                                   |
| <code>if rcv_base &lt;= seq &lt; rcv_base + window: # inside our receive window</code>                                                            | Is the packet inside our receive window? (i.e. new and not too far ahead.)                                     |
| <code>buffer[seq] = (packet_type, payload) # keep it (even if out of order)</code>                                                                | Keep it in the buffer, even if it arrived out of order. (This is what GBN could not do.)                       |
| <code>chan.sendto(make_packet(ACK, seq), dest) # ACK this exact packet</code>                                                                     | Send an ACK for exactly this packet number.                                                                    |
| <code>while rcv_base in buffer: # deliver everything now in order</code>                                                                          | Now deliver as many in-order packets as possible: as long as the packet we need next is in the buffer.         |
| <code>stored_type, stored_payload = buffer.pop(rcv_base)</code>                                                                                   | Take it out of the buffer.                                                                                     |
| <code>if stored_type == FIN:<br/>    finished = True<br/>else:<br/>    chunks.append(stored_payload)</code>                                       | FIN -> we are done. DATA -> append to the file pieces.                                                         |
| <code>rcv_base += 1</code>                                                                                                                        | Move on to the next packet number.                                                                             |
| <code>elif seq &lt; rcv_base: # old packet: our ACK was lost</code>                                                                               | The packet is OLD: we already delivered it. That means our ACK for it was lost (or the network duplicated it). |
| <code>chan.sendto(make_packet(ACK, seq), dest) # so ACK again</code>                                                                              | ACK it again so the sender can stop resending it.                                                              |

## Viva questions you may be asked (with easy answers)

### Q. What is the main difference between GBN and SR?

A. GBN: cumulative ACKs, no receiver buffer, resend whole window on timeout. SR: individual ACKs, receiver buffer, one timer per packet, resend only the lost packet.

### Q. Why does the SR receiver need a buffer?

A. So that out-of-order packets are not thrown away; they wait until the missing earlier packet arrives and then all get delivered in order.

### Q. Why must the receiver window equal the sender window in SR?

A. If the receiver's window were different, sequence numbers could be mistaken for old or new packets. Both use the same `window` value in our code.

### Q. Explain Jacobson/Karels.

A. Keep srtt (smoothed RTT) and rttvar (smoothed deviation).  $rttvar = 0.75 * rttvar + 0.25 * [srtt - sample]$ ;  $srtt = 0.875 * srtt + 0.125 * sample$ ;  $RTO = srtt + 4 * rttvar$ .

### Q. Explain Karn's algorithm.

A. Two rules: don't use RTT samples from retransmitted packets (ambiguous which copy was acked), and double the RTO after a timeout (backoff).

### Q. Why is the RTT sample ambiguous for retransmitted packets?

A. If we send a packet twice and then get one ACK, we can't tell whether it answers the first or the second copy, so the measured time could be much too big or too small.

### Q. Why is $RTO = srtt + 4 * rttvar$ and not just srtt?

A. Half the time the real RTT would be above the average and we'd time out for nothing. The  $4 * rttvar$  margin covers normal variation.

### Q. Why did you turn adaptive RTO off in the experiments?

A. The claim is about protocol design. With the same fixed timeout for all three, the difference in goodput comes only from the protocol, which is fair. RTO tuning is a control, as the proposal says.

### Q. What happens on a timeout in your SR sender?

A. Only packets whose own timer expired are resent, their timers restart, they are marked as resent (Karn), and the RTO is doubled once.

### Q. Why is SR better with higher loss and bigger windows?

A. It never resends packets that arrived fine, so no bandwidth is wasted, and the pipe remains full since ACKed packets slide the window. GBN wastes more as the window and the loss grow.

**Q. What are the limits of SR?**

A. The receiver needs memory for the buffer and the sender needs a timer per packet, so it is more complex than GBN.

**Q. Why  $\max(0.001, \dots)$  when computing wait?**

A. If the earliest timer has already expired the difference would be zero or negative; a settimeout of 0 means non-blocking mode in sockets, so we use at least 1 ms.